南京大学
NANJING UNIVERSITY

数据结构
Data Structure

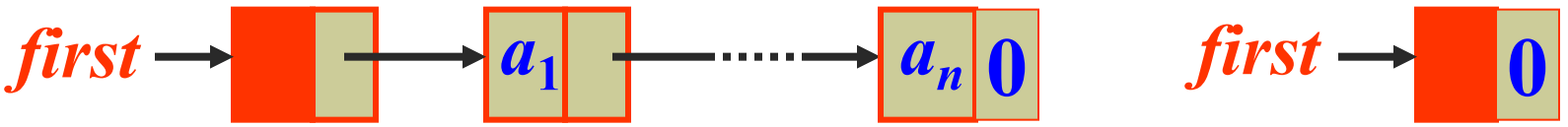
许 蕾

xlei@nju.edu.cn

南京大学计算机学院

2.3.4 带附加头结点（表头结点）的单链表

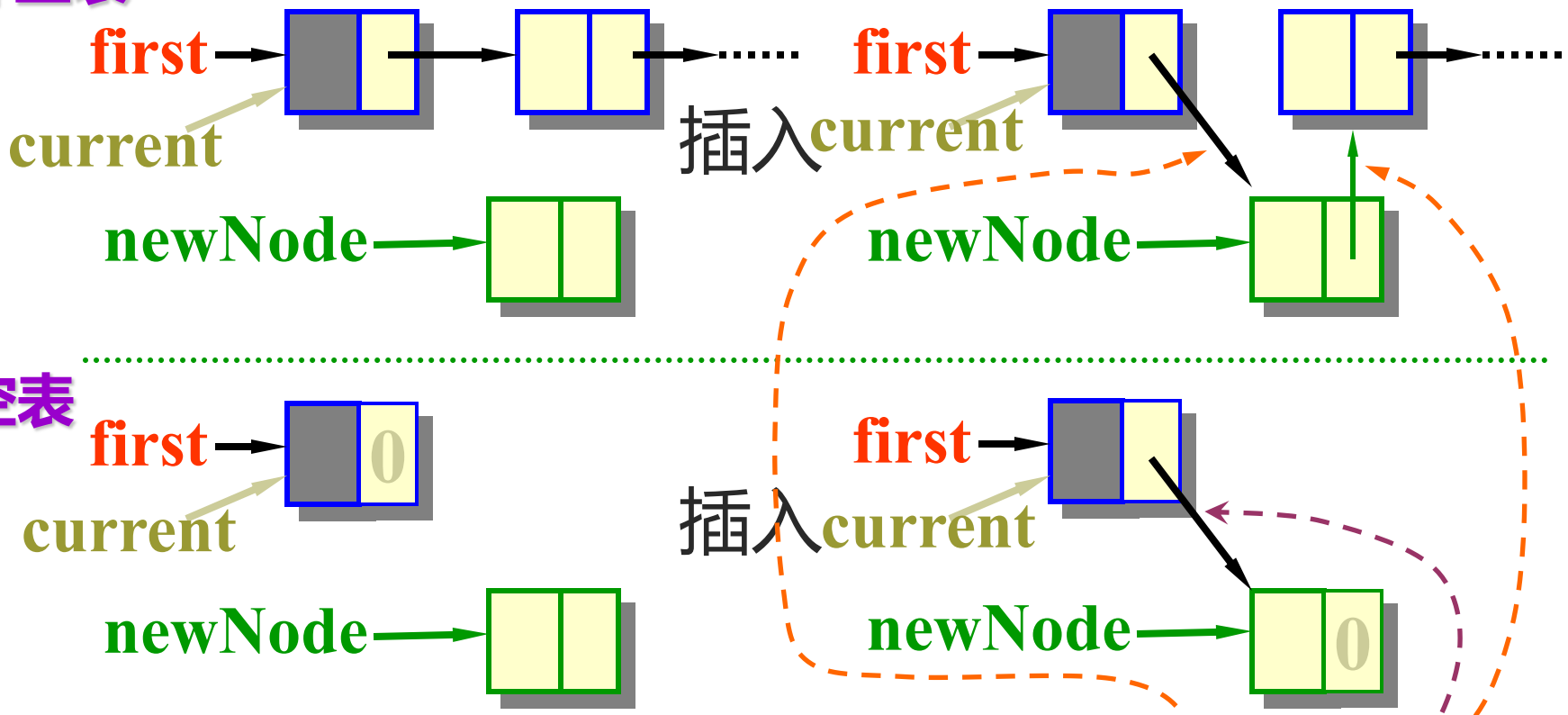
- 设置表头结点的目的是统一空表与非空表的操作，简化链表操作的实现。
- 表头结点位于表的最前端，本身不带数据，仅标志表头。



非空表　　空表

在带表头结点的单链表最前端插入新结点

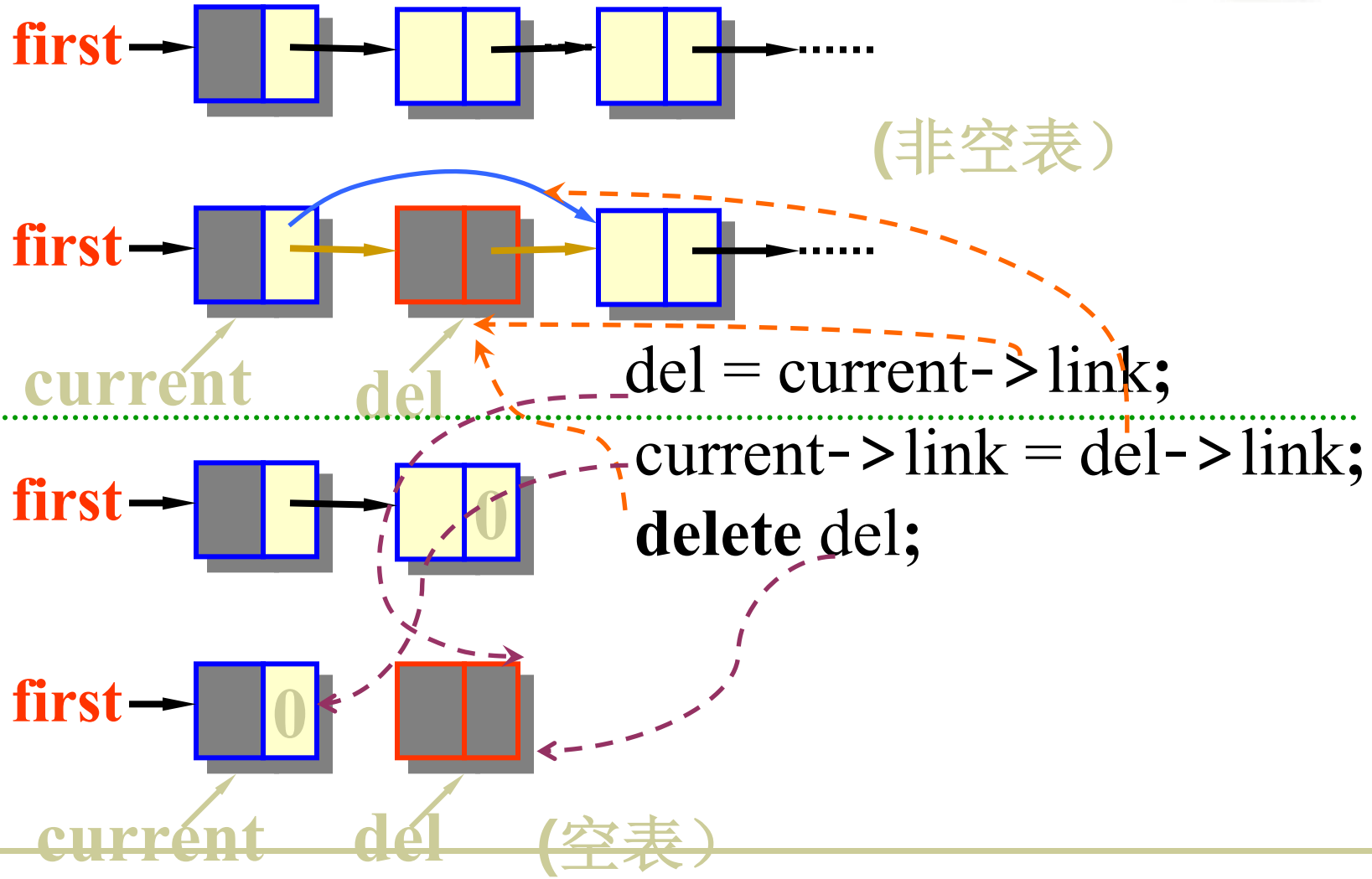
非空表



空表

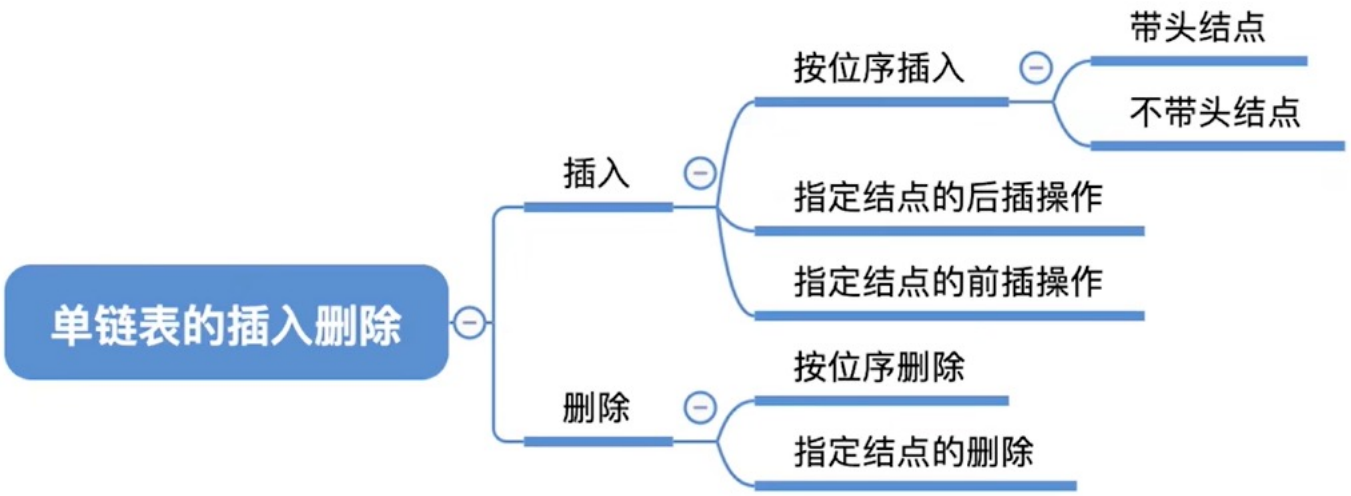

```
newNode->link = current->link;
current->link = newNode;
```

从带表头结点的单链表中删除最前端的结点



单链表的局限性：

无法逆向检索，有时候不太方便



单链表的查找

单链表的创建p65-66

- 尾插法（设置指向表尾的指针）
- 头插法（链表的逆置）

§2.4 线性链表的其他变形

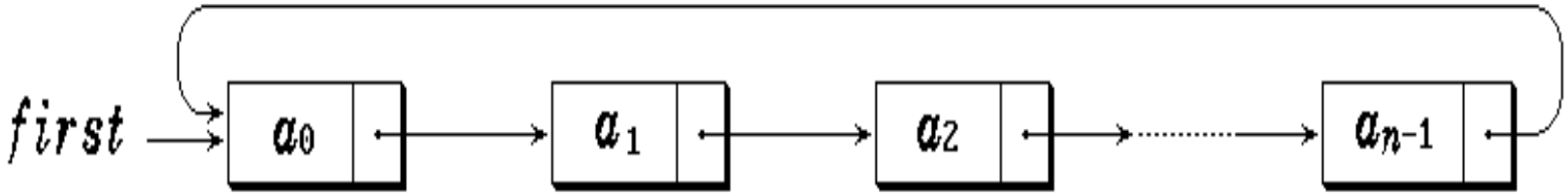
- 循环链表
- 双向链表

单链表

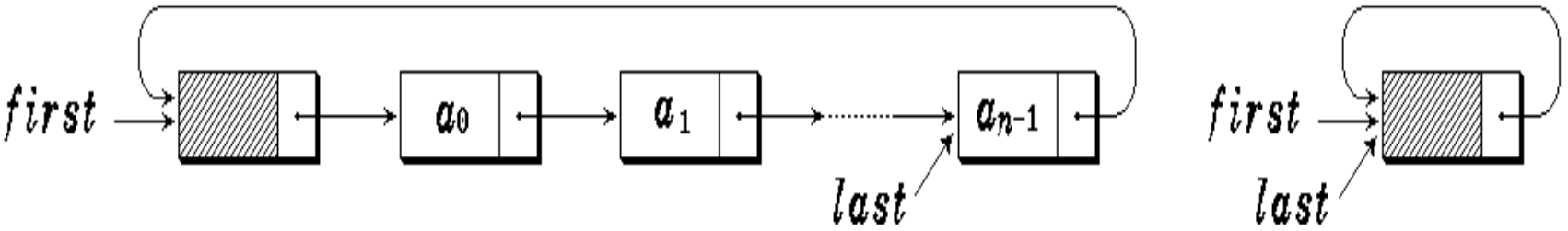
循环链表

双向链表

2.4.1 循环链表 (*Circular List*)



循环链表



带表头结点的循环链表

循环链表的特点：

- 循环链表最后一个结点的 *link* 指针不 为 0 (*NULL*)，而是指向了表的前端。

- 只要知道表中某一结点的地址，就可搜寻到所有其他结点的地址。

循环链表类的定义

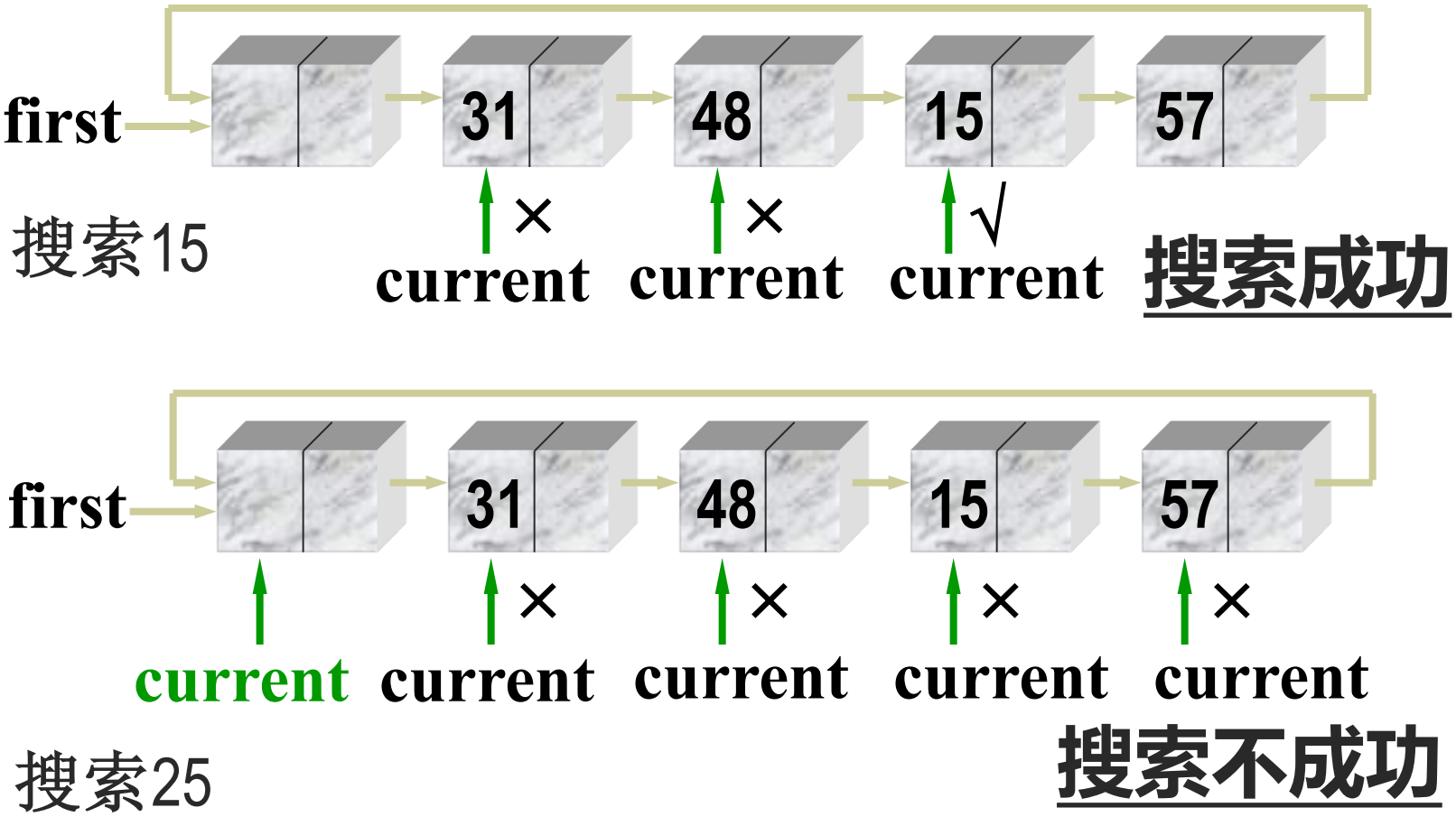
```
template <class E>
struct CircLinkNode {                    //链表结点类定义
    E data;
    CircLinkNode<E> *link;
    CircLinkNode ( CircLinkNode<E> *next =
        NULL ) { link = next; }
    CircLinkNode ( E x, CircLinkNode<E> *next =
        NULL ) { data = x;  link = next; }
    bool Operator==(CircLinkNode<E> & node) {
        return data == node.data; }
    bool Operator!=(CircLinkNode<E> & node) {
        return data != node.data; }
};
```

```
template <class E>      //链表类定义
class CircList : public LinearList<E> {
private:
    CircLinkNode<E> *first, *last;  //头指针, 尾指针
public:
    CircList(const E x);                    //构造函数
    CircList(CircList<E>& L);               //复制构造函数
    ～CircList();                           //析构函数
   int Length() const;                      //计算链表长度
    bool IsEmpty() { return first->link == first; }
                                            //判表空否
    CircLinkNode<E> *getHead() const;
                                  //返回表头结点地址
```

```
    void setHead ( CircLinkNode<E> *p );
                                        //设置表头结点地址
    CircLinkNode<E> *Search ( E x );          //搜索
    CircLinkNode<E> *Locate ( int i );        //定位
    E *getData ( int i );                     //提取
    void setData ( int i, E x );              //修改
     bool Insert ( int i, E x );              //插入
    bool Remove ( int i, E& x);               //删除
};
```

- **循环链表与单链表的操作实现，最主要的不同就是扫描到链尾，遇到的不是NULL，而是表头。**

循环链表的搜索算法

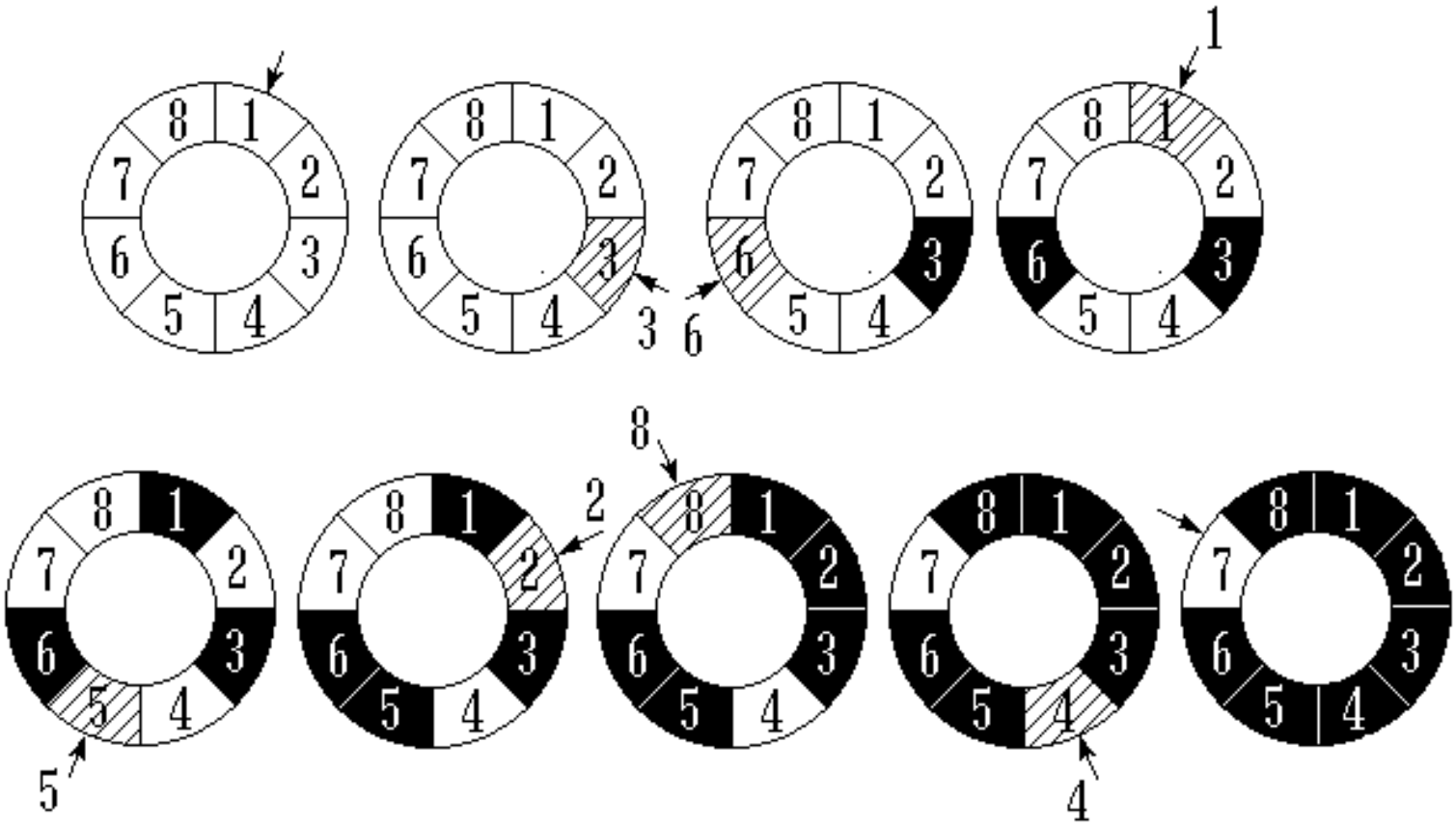

first
31
48
15
57
搜索15
current
current
current
搜索成功
first
31
48
15
57
current
current
current
current
current
搜索25
搜索不成功

循环链表的搜索算法

```
template <class E>
CircListNode<E> * CircList<E>::Search( E x )
{
//在链表中从头搜索其数据值为 x 的结点
    current = first->link;
    while ( current != first && current->data != x )
        current = current->link;
    return current;
}
```

循环链表示例：求解约瑟夫问题

- 约瑟夫问题 （例：$n=8 \quad m=3$）



求解Josephus问题的算法

```
#include <iostream.h>
#include "CircList.h"
template <class E>
void Josephus(CircList<E>& Js, int n, int m) {
    CircLinkNode<E> *p = Js.getHead(),
                    *pre = NULL;
    int i, j;
    for ( i = 0; i < n-1; i++ ) {          //执行n-1次
        for ( j = 1; j < m; j++)           //数m-1个人
            { pre = p;  p = p->link; }
        cout << "出列的人是"  << p->data << endl;
```

```
            pre->link = p->link;  delete p;          //删去
            p = pre->link;
        }
};

void main() {
    CircList< int> clist;
    int i, n, m;
    cout << "输入游戏者人数和报数间隔 : ";
    cin >> n >> m;
    for (i = 1; i <= n; i++ ) clist.insert(i, i);    //约瑟夫环
    Josephus(clist, n, m);                           //解决约瑟夫问题
}
```

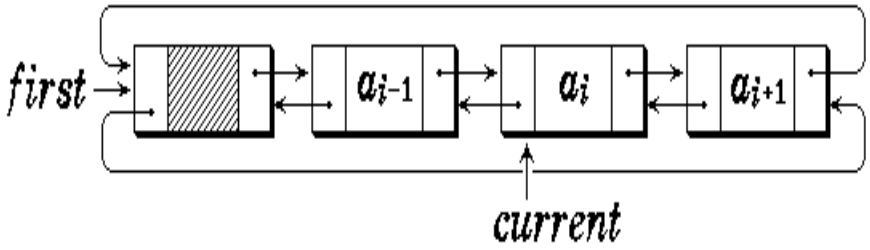
2.4.2 双向链表（doubly linked list）

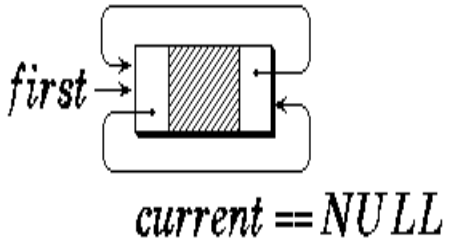
- 每个结点的结构

lLink (左链指针)	*data* (数据)	*rLink* (右链指针)

前驱方向 ← → 后继方向

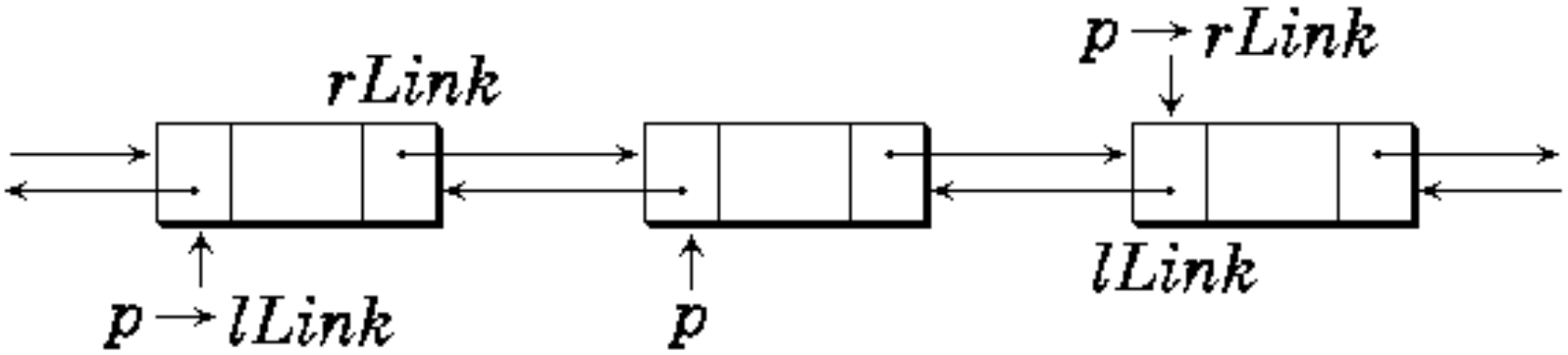
- 带头结点的双向(循环)链表表示





- 链表中的结点指针关系

$$p == p\rightarrow lLink\rightarrow rLink == p\rightarrow rLink\rightarrow lLink$$



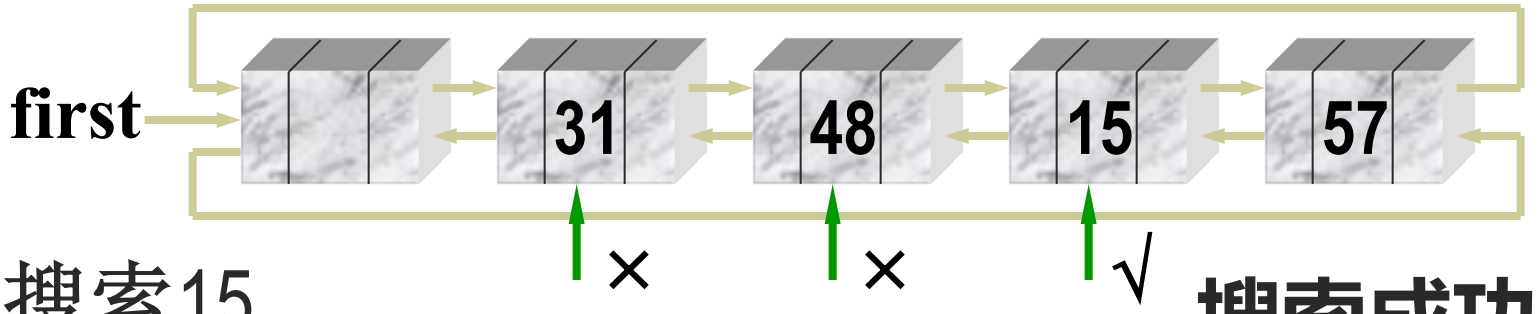
双向循环链表类的定义

```
template <class E>
struct DblNode {                    //链表结点类定义
    E data;                         //链表结点数据
    DblNode<E> *lLink, *rLink;  //前驱、后继指针
    DblNode ( DblNode<E> *l = NULL, DblNode<E> *r =
     NULL )
       { lLink = l;  rLink = r; }                    //构造函数
    DblNode ( E value, DblNode< E> *l = NULL,DblNode<E>
     *r = NULL)
       { data = value;  lLink = l;  rLink = r; }  //构造函数
};
```

```
template <class E>
class DblList {                                    //链表类定义
   public:
      DblList ( E uniqueVal ) {                    //构造函数
         first = new DblNode<E> (uniqueVal);
         first->rLink = first->lLink = first;
      };
      DblNode<E> *getFirst () const { return first; }
      void setFirst ( DblNode<E> *ptr ) { first = ptr; }
      DblNode<E> *Search ( E x, int d);
//在链表中按d指示方向寻找等于给定值x的结点,
//d=0按前驱方向, d≠0按后继方向
```

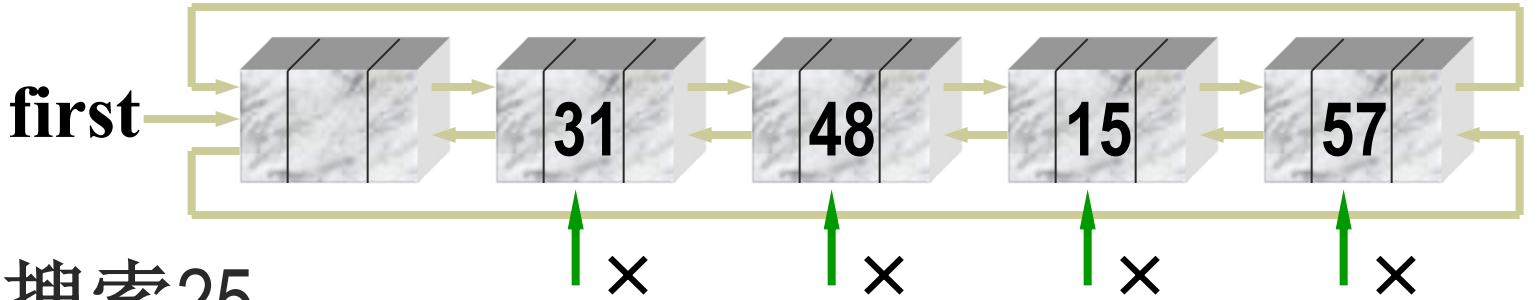
```
  DblNode<E> *Locate ( int i, int d );
//在链表中定位序号为i(≥0)的结点, d=0按前驱方向,
//d≠0按后继方向
  bool Insert ( int i, E x, int d );
//在第i个结点后插入一个包含有值x的新结点,
//d=0按前驱方向,d≠0按后继方向
  bool Remove ( int i, E& x, int d );   //删除第i个结点
  bool IsEmpty() { return first->rlink == first; }
//判双链表空否
private:
  DblNode<E> *first;                    //表头指针
};
```

双向循环链表的搜索算法



搜索15

搜索成功



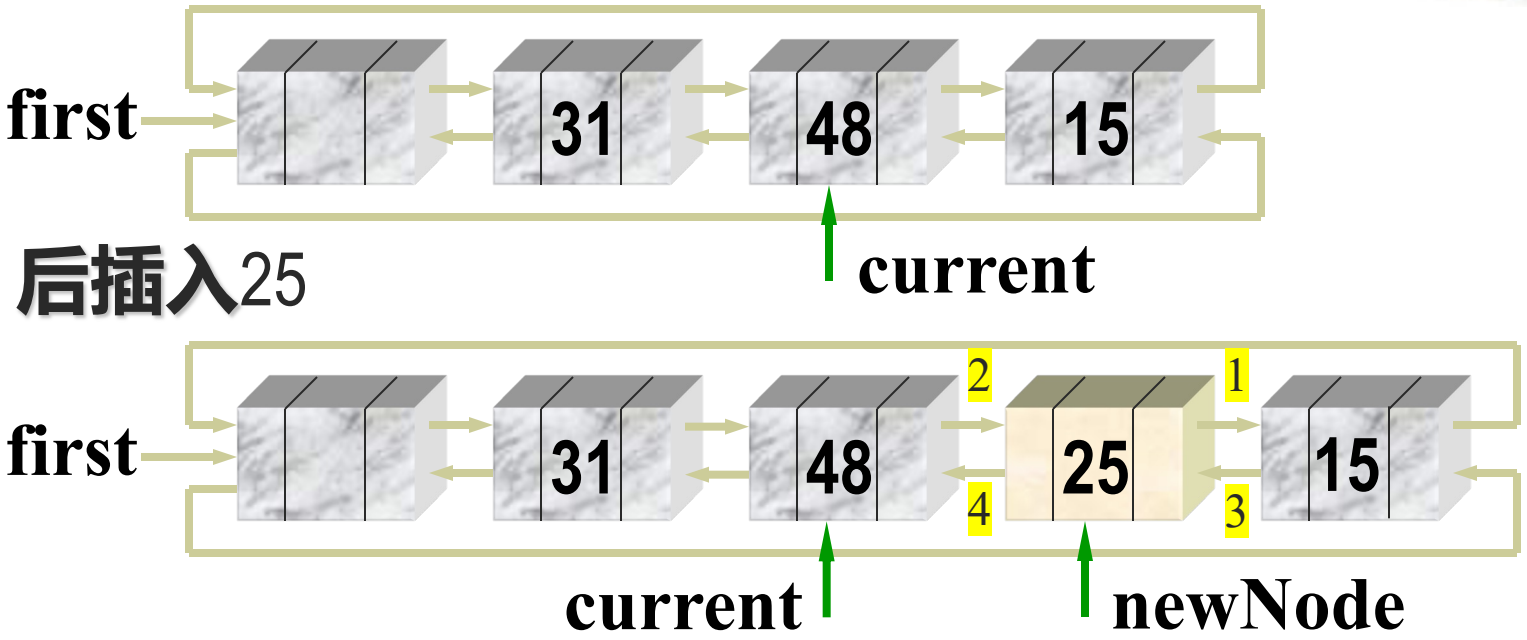
搜索25

搜索不成功

双向循环链表的搜索算法

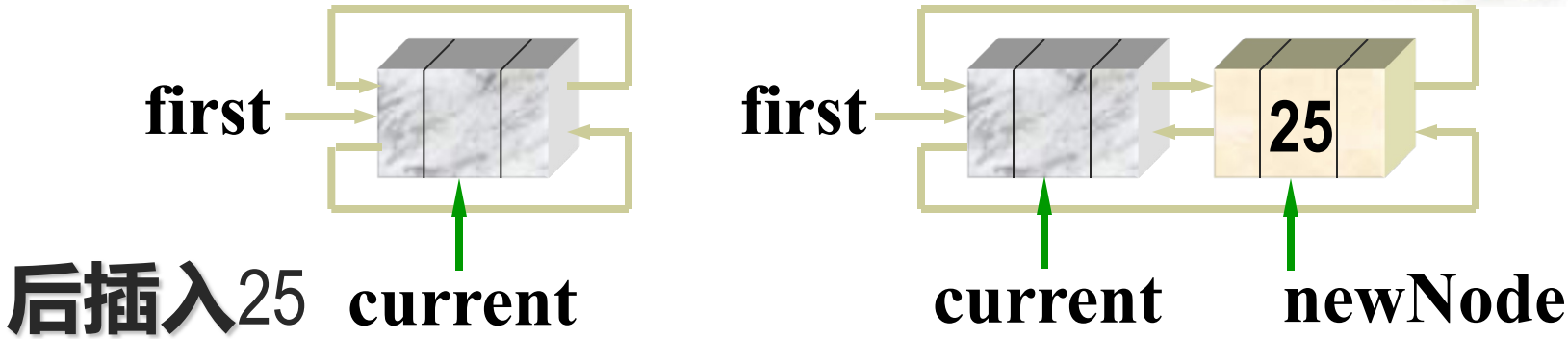
```
template <class E>
DblNode< E> *DblList<E>::Search (E x, int d) {
//在双向循环链表中寻找其值等于x的结点。
   DblNode<E> *current = (d == 0)?
        first->lLink : first->rLink;   //按d确定搜索方向
    while ( current != first && current->data != x )current
    = (d == 0) ?
          current->lLink : current->rLink;
    if ( current != first ) return current;  //搜索成功
    else return NULL;                        //搜索失败
};
```

双向循环链表的插入算法（非空表）




```
newNode->rLink = current->rLink;
current->rLink = newNode;
newNode->rLink->lLink = newNode;
newNode->lLink = current;
```

双向循环链表的插入算法（空表）



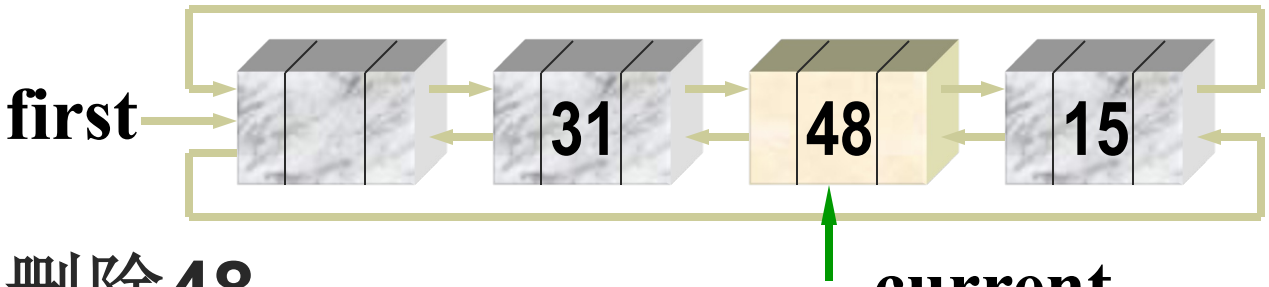

```
newNode->rLink = current->rLink
      (newNode->rLink = first);
current->rLink = newNode;
newNode->rLink ->lLink = newNode;
              ( first->lLink = newNode )
newNode->lLink = current;
```

双向循环链表的插入算法

```
template <class E>
bool DblList<E>::Insert ( int i, E x, int d ) {
//建立一个包含有值x的新结点, 并将其按 d 指定的
//方向插入到第i个结点之后。
    DblNode< E> *current = Locate(i, d);
      //按d指示方向查找第i个结点
   if ( current == NULL ) return false;    //插入失败
    DblNode<E> *newNd = new DblNode<E>(x);
    if (d == 0) {       //前驱方向:插在第i个结点左侧
       newNd->lLink = current->lLink; //链入lLink链
       current->lLink = newNd;
```

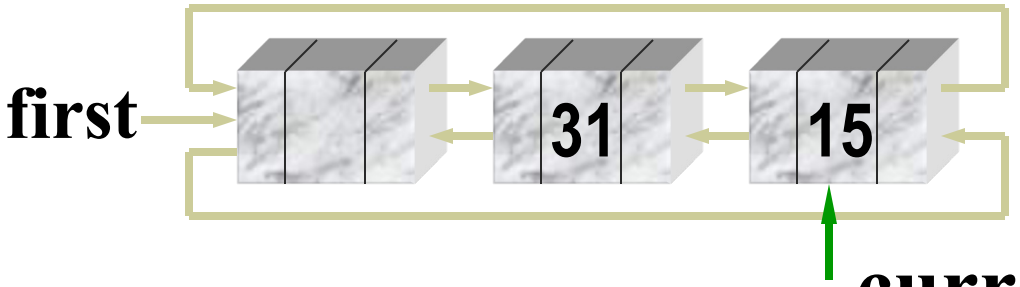
```
        newNd->lLink->rLink = newNd;  //链入rLink链
        newNd->rLink = current;
    } else {                          //后继方向:插在第i个结点后面
        newNd->rLink = current->rLink;  //链入rLink链
        current->rLink = newNd;
        newNd->rLink->lLink = newNd;   //链入lLink链
        newNd->lLink = current;
    }
    return true;          //插入成功
};
```

双向循环链表的删除算法



非空表

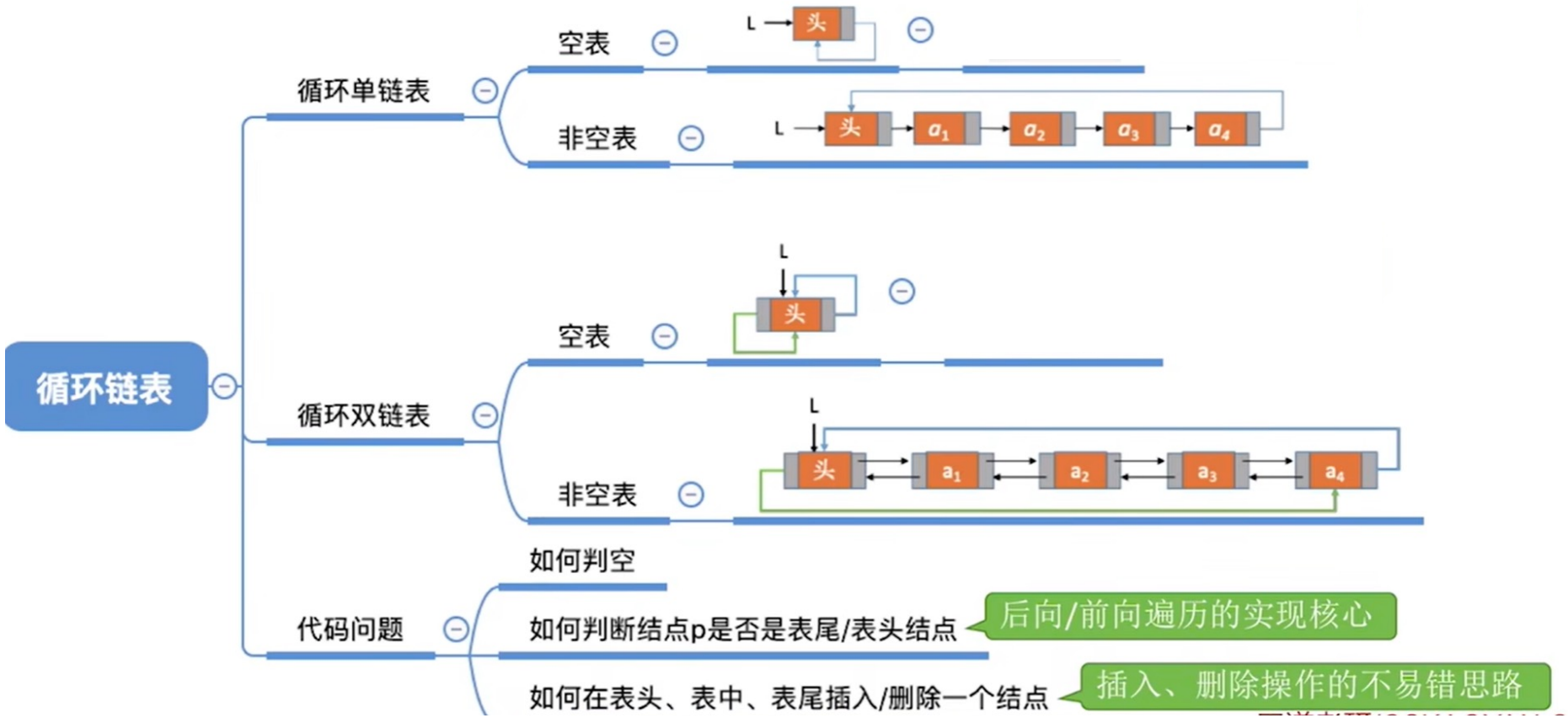
删除48




```
current->rLink->lLink = current->lLink;
current->lLink->rLink = current->rLink;
```

双向循环链表的删除算法

```
template < class E>
bool DblList< E>::Remove( int i, E& x, int d ) {
//在双向循环链表中按d所指方向删除第i个结点。
   DblNode< E> *current = Locate (i, d);
   if (current == NULL) return false;        //删除失败
    current->rLink->lLink = current->lLink;
    current->lLink->rLink = current->rLink;
        //从lLink链和rLink链中摘下
   x = current->data;  delete current;        //删除
   return true;                               //删除成功
};
```


循环链表
循环单链表
空表
L
头
非空表
L
头
a_1
a_2
a_3
a_4
循环双链表
空表
L
头
非空表
L
头
a_1
a_2
a_3
a_4
代码问题
如何判空
如何判断结点p是否是表尾/表头结点
后向/前向遍历的实现核心
如何在表头、表中、表尾插入/删除一个结点
插入、删除操作的不易错思路

2.5 多项式及其运算

$$A(x) = 1 - 10x^6 + 2x^8 + 7x^{14}$$

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

$$= \sum_{i=0}^{n} a_i x^i$$

- n阶多项式 $P_n(x)$ 有 $n+1$ 项。
  - 系数 $a_0, a_1, a_2, \ldots, a_n$
  - 指数 0, 1, 2, …, n按升幂排列

多项式的顺序存储表示

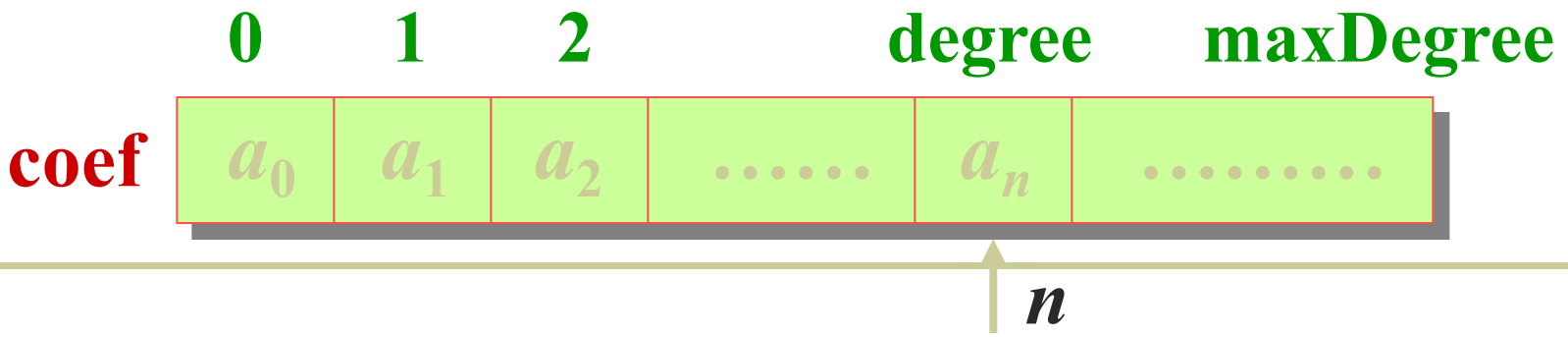
第一种：(静态数组表示)

```
private:
    int degree;
    float coef [maxDegree+1];
```

$P_n(x)$可以表示为：

pl.degree = n

pl.coef[i] = a_i, $0 \le i \le n$



第二种：(动态数组表示)

```
private:
    int degree;
    float * coef;
        Polynomial :: Polynomial (int sz) {
            maxDegree = sz;
            coef = new float [maxDegree + 1];
        }
```

- 以上两种存储表示适用于指数连续排列的多项式。但对于绝大多数项的系数为零的多项式，如 $P_{101}(x) = 3+5x^{50}-4x^{101}$, 不经济。

第三种：适用于稀疏多项式

	0	1	2		i		m
coef	a_0	a_1	a_2	……	a_i	……	a_m
exp	e_0	e_1	e_2	……	e_i	……	e_m

```
struct term {          //多项式的项定义
        float coef;          //系数
        int exp;             //指数
};

static term termArray[maxTerms];     //项数组
static int free, maxTerms;           //当前空闲位置指针
```

初始化：

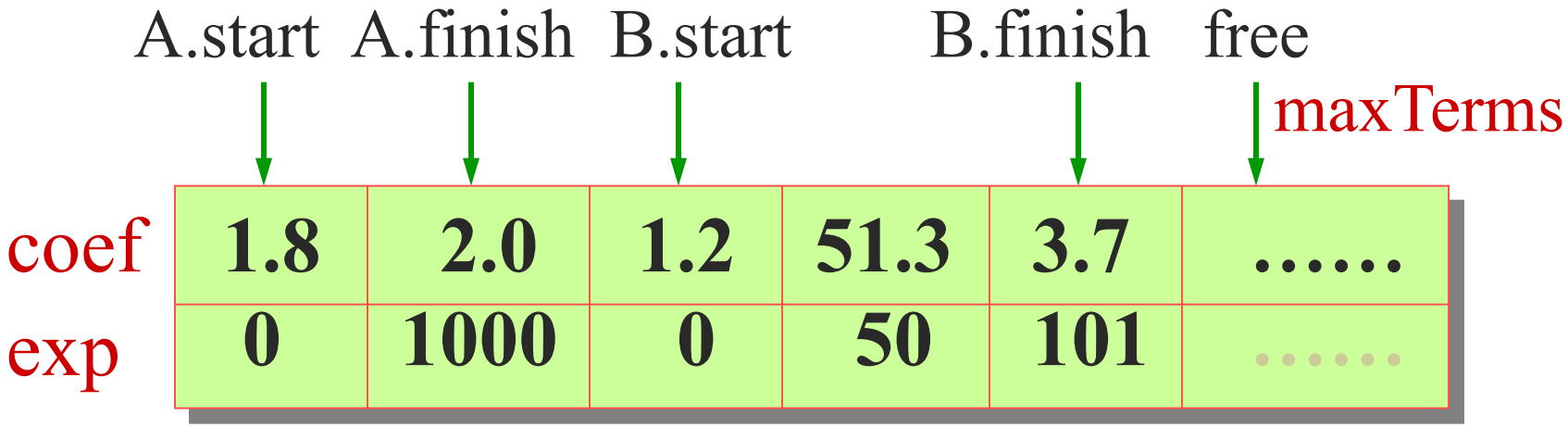
```
// term Polynomial::termArray[maxTerms];
// int Polynomial::free = 0;

class Polynomial {          //多项式定义
   public:
      ……
   private:
      int start, finish;        //多项式始末位置
}
```

两个多项式存储的例子

$A(x) = 1.8+2.0x^{1000}$

$B(x) = 1.2 + 51.3x^{50} + 3.7x^{101}$

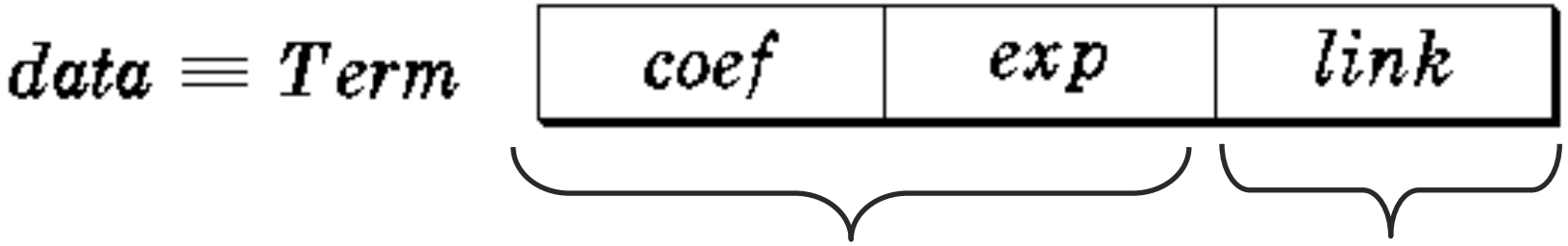


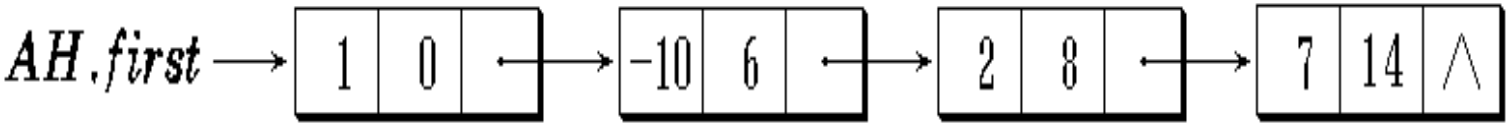
两个多项式存放在termArray中

第四种：多项式的链表存储表示

$$A(x) = 1 - 10x^6 + 2x^8 + 7x^{14}$$

每个结点的结构:





多项式(polynomial)类的链表定义

```
struct Term {                          //多项式结点定义
    float coef;                        //系数
    int exp;                           //指数
    Term *link;                        //链接指针
    Term (float c, int e, Term *next = NULL)
       { coef = c;  exp = e;  link = next;}
     Term *InsertAfter ( float c, int e);
    friend ostream& operator << (ostream&,
                const Term& );
};
```

```
class Polynomial {                          //多项式类的定义
public:
    Polynomal() { first = new Term(0, -1); } //构造函数
    Polynomal (Polynomal& R);               //复制构造函数
    int maxOrder();                         //计算最大阶数
private:
    Term *first;
   friend ostream& operator << (ostream&,
        const Polynomal& );
   friend istream& operator >> ( istream&,
       Polynomal& );
```

```
    friend void Add ( Polynomial& A, Polynomial& B,
                Polynomial& C );
    friend void Mul ( Polynomial& A, Polynomial& B,
                Polynomial& C );
};
```

- 多项式顺序存储表示的缺点
  - 插入和删除时项数可能有较大变化，因此要移动大量数据
  - 不利于多个多项式的同时处理

- 多项式链表存储表示的优点
  - 多项式的项数可以动态地增长，不存在存储溢出问题
  - 插入、删除方便，不移动元素

两个多项式的相加算法

- 设两个多项式都带表头结点，检测指针pa和pb分别指示两个链表当前检测结点，并设结果多项式的链表为C，存放指针为pc，初始位置在C的表头结点。

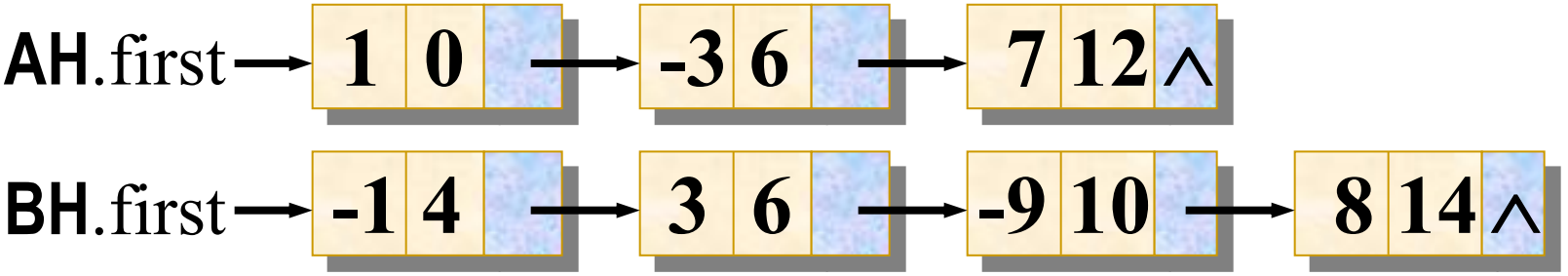
- 当pa和pb没有检测完各自的链表时，比较当前检测结点的指数域：
  - 指数不等：小者加入C链，相应检测指针pa或者pb进1；
  - 指数相等：对应项系数相加。若相加结果不为零，则结果加入C链，pa与pb进1。
  - 当pa或pb指针中有一个为NULL，则把另一个链表的剩余部分加入到C链。

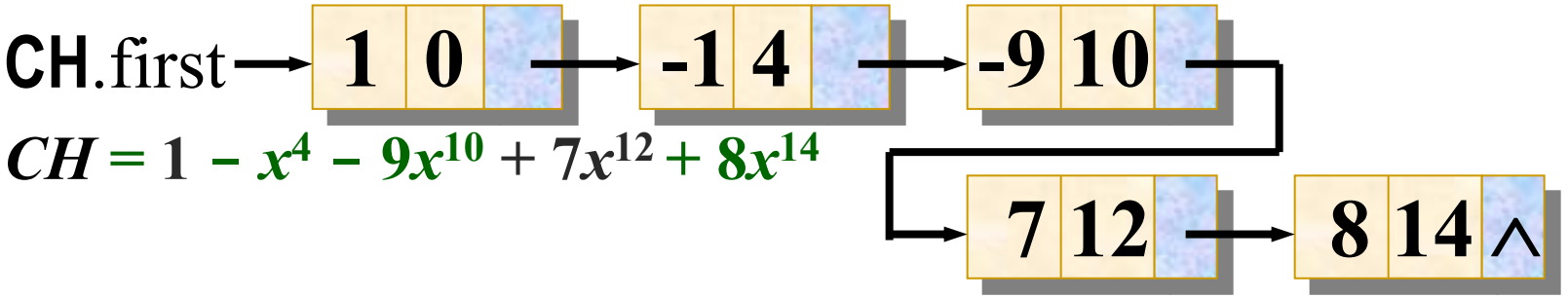
书本P78程序2.30

多项式链表的相加

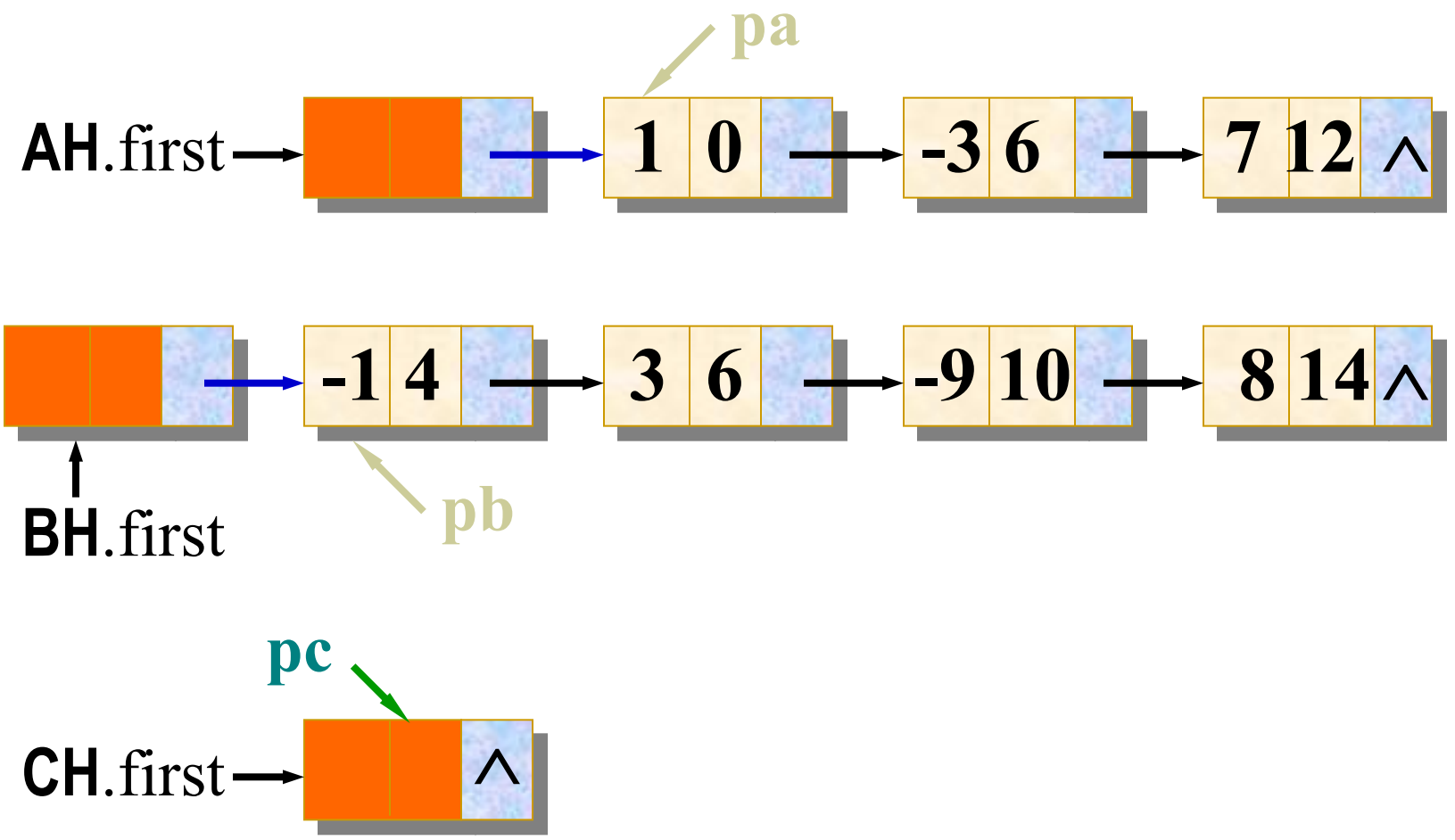
$AH = 1 - 3x^6 + 7x^{12}$

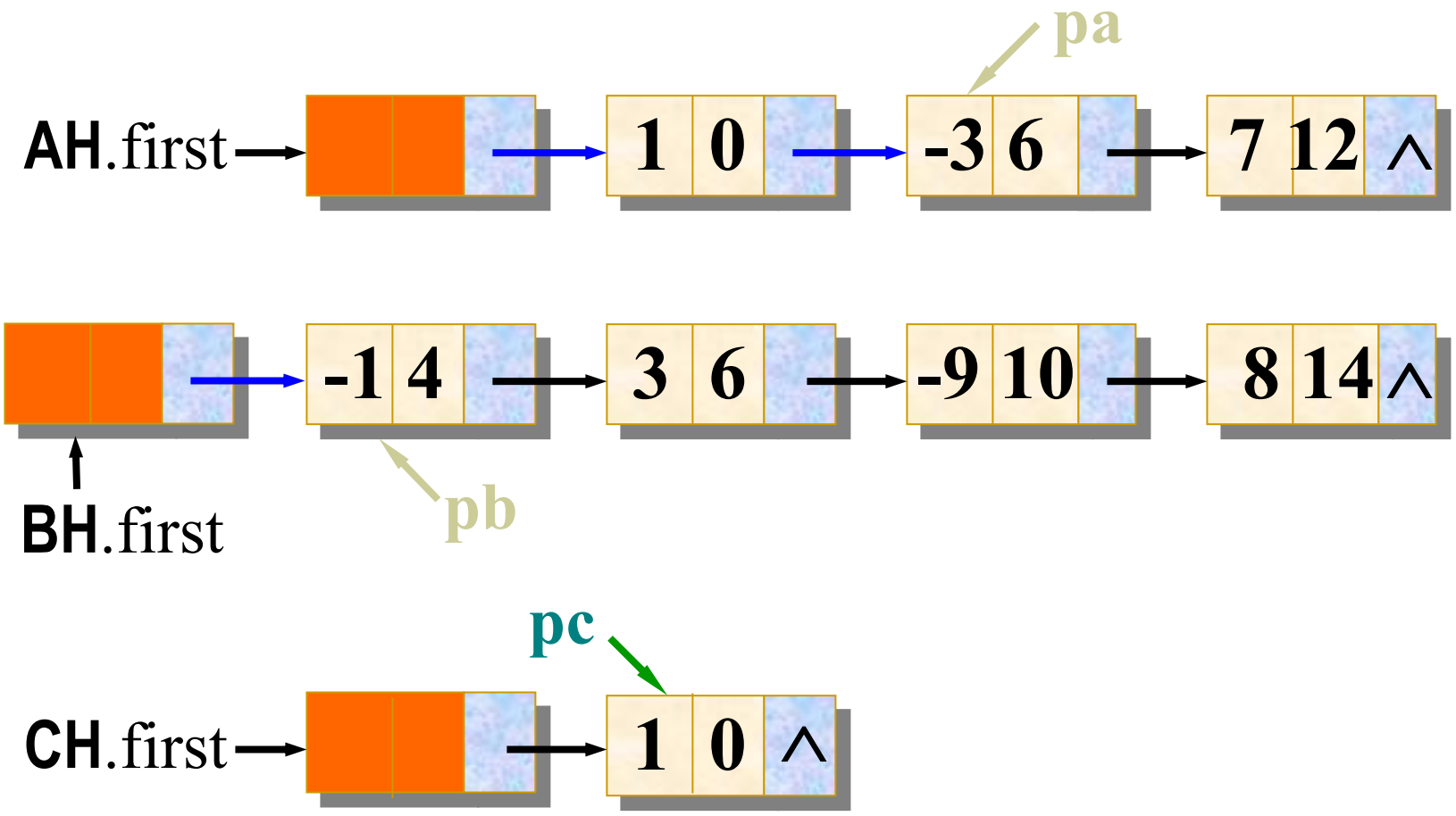
$BH = -x^4 + 3x^6 - 9x^{10} + 8x^{14}$

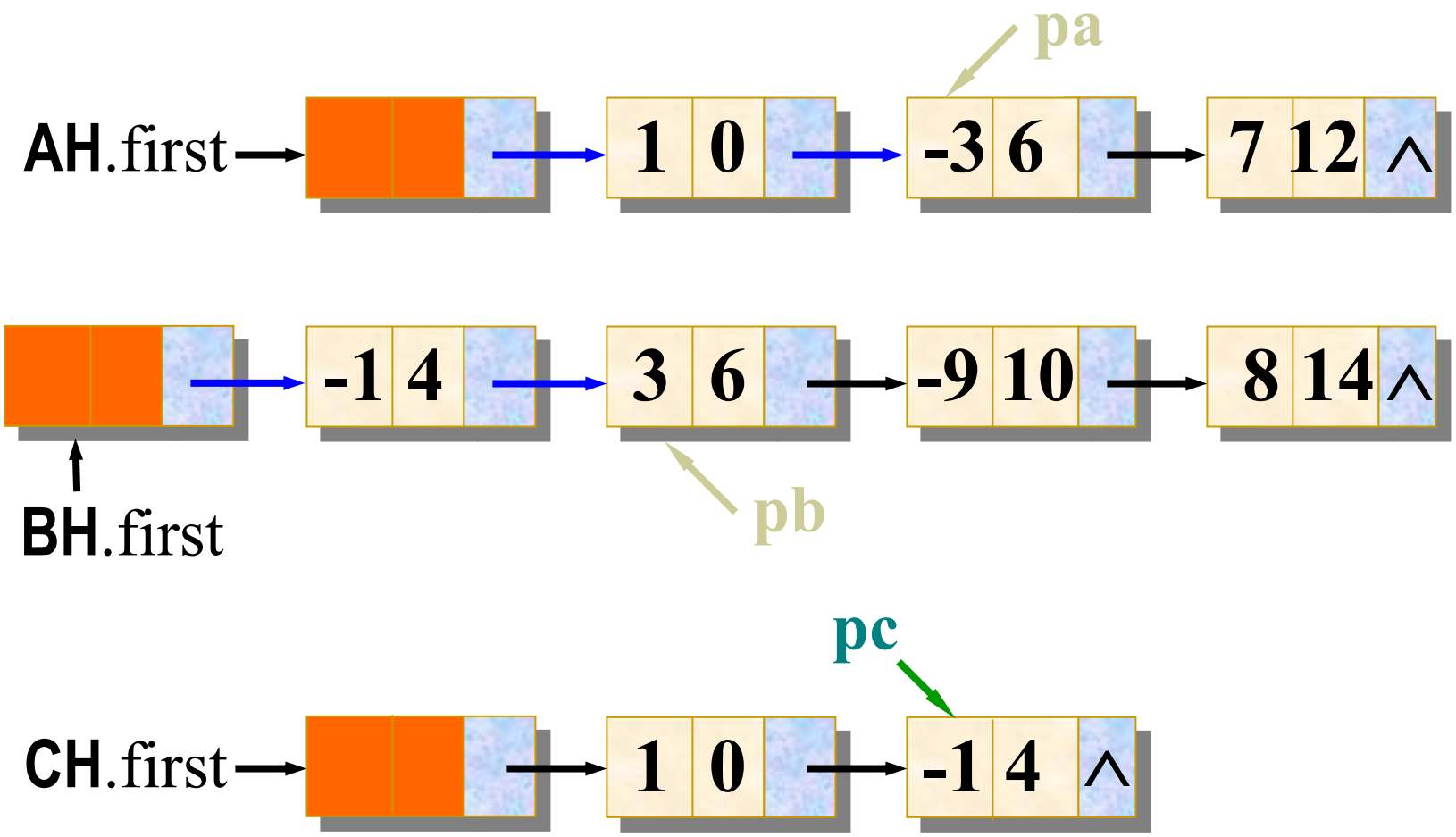


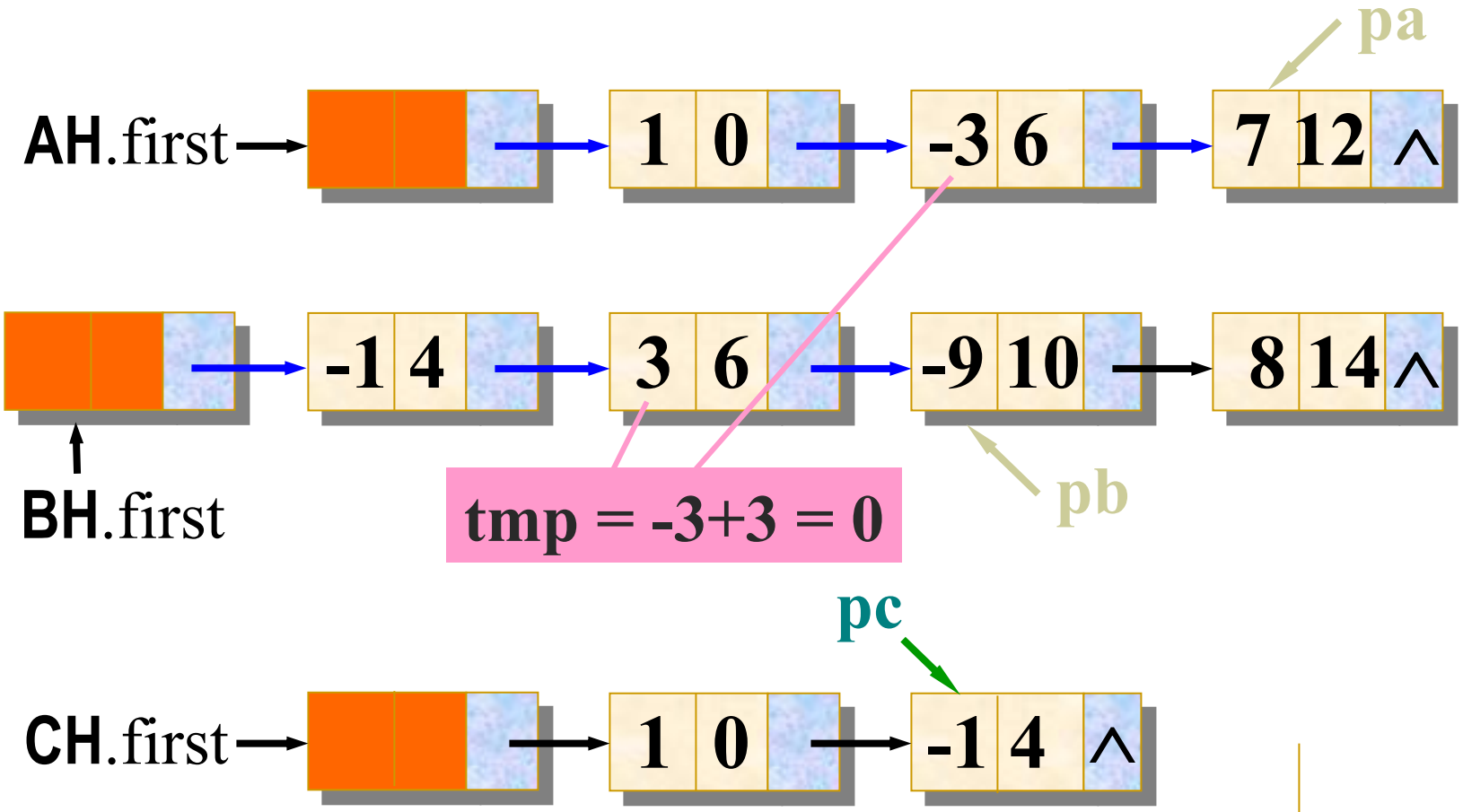


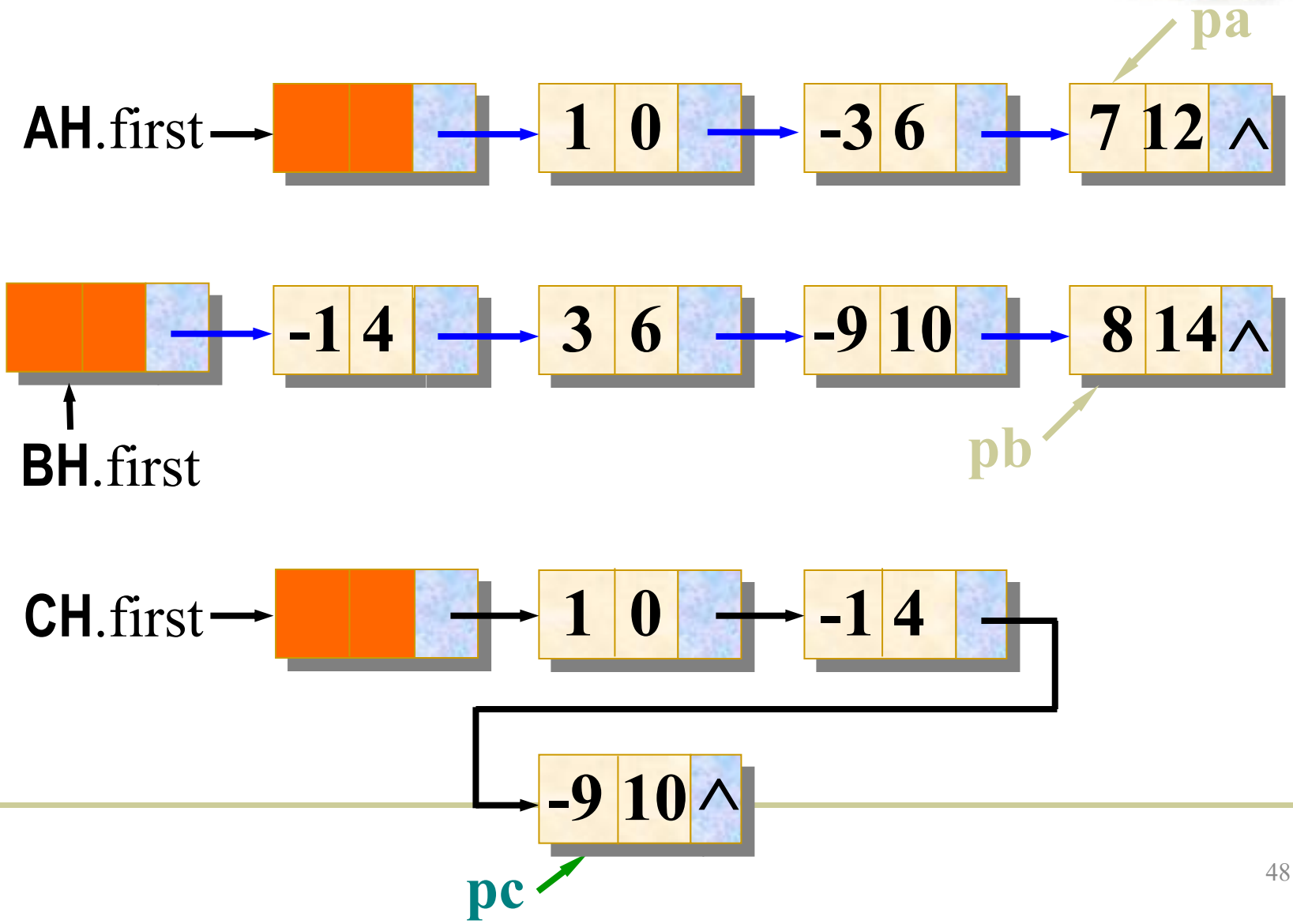
$CH = 1 - x^4 - 9x^{10} + 7x^{12} + 8x^{14}$

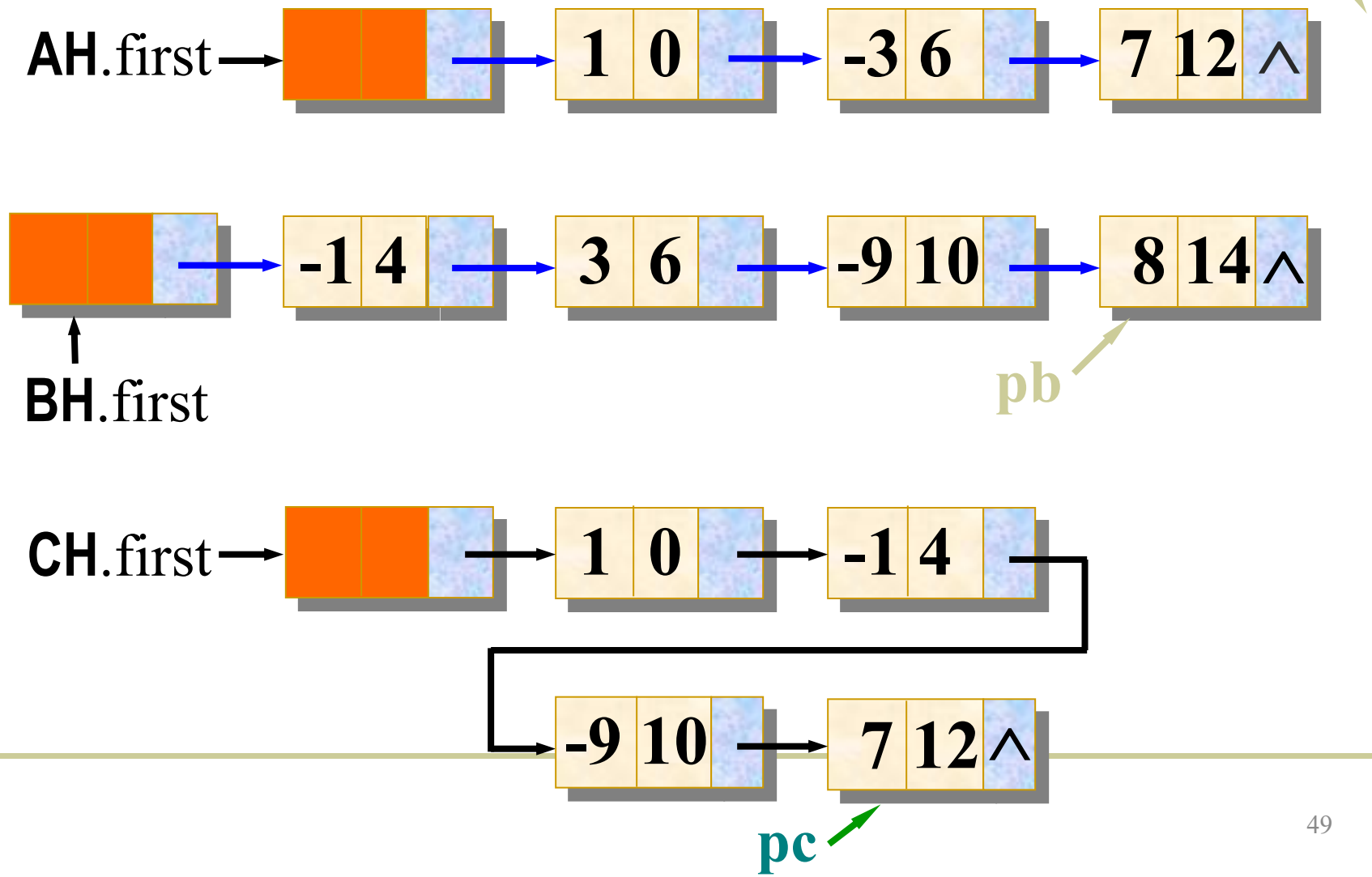

pa
AH.first
1 0
-3 6
7 12 ∧
-1 4
3 6
-9 10
8 14 ∧
pb
BH.first
pc
CH.first
∧

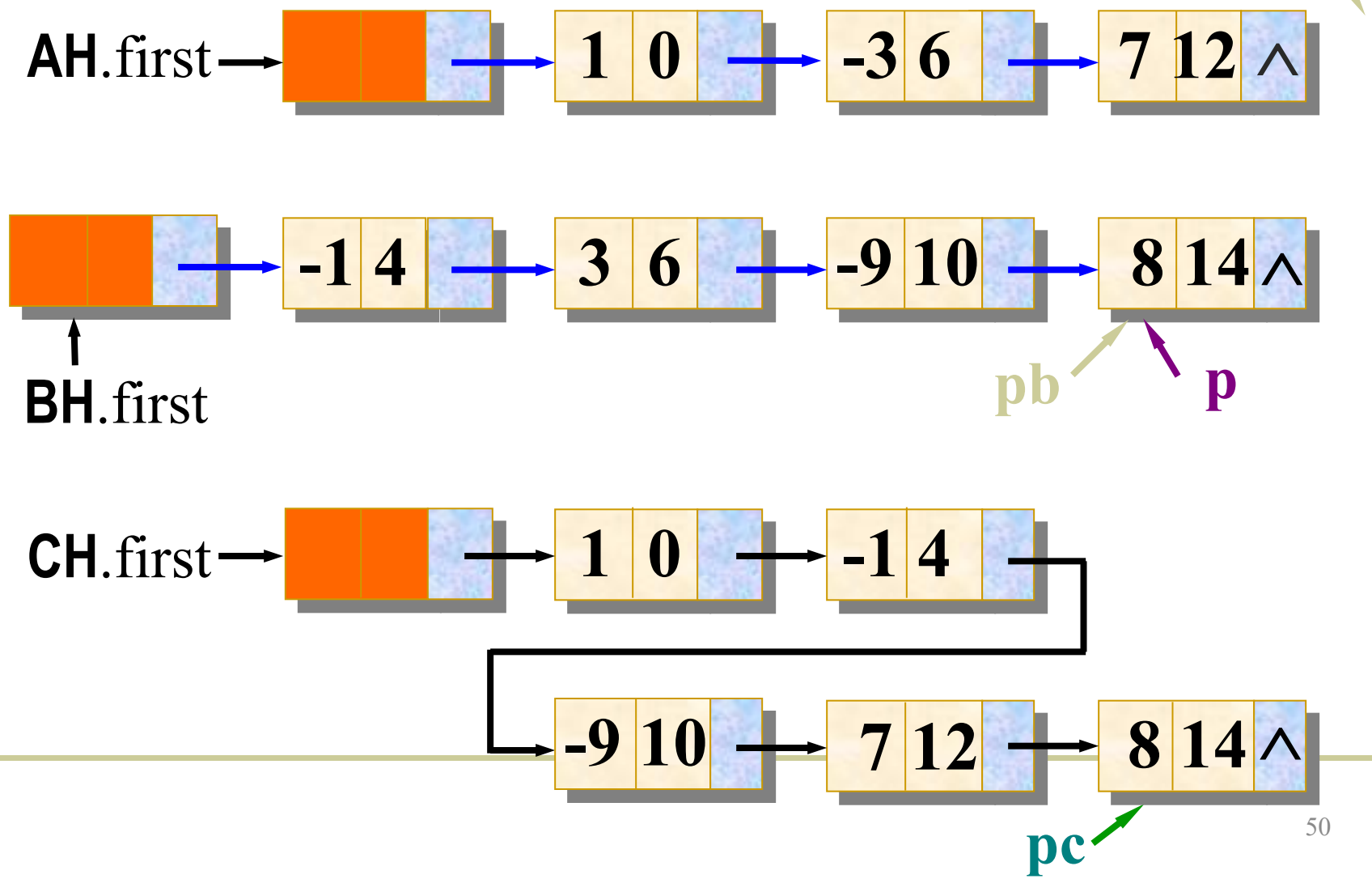

pa
AH.first
1 0
-3 6
7 12 ∧
-1 4
3 6
-9 10
8 14 ∧
pb
BH.first
pc
CH.first
1 0 ∧


pa
AH.first
1 0
-3 6
7 12 ∧
-1 4
3 6
-9 10
8 14 ∧
pb
BH.first
pc
CH.first
1 0
-1 4 ∧


pa
AH.first
1 0
-3 6
7 12 ∧
-1 4
3 6
-9 10
8 14 ∧
BH.first
tmp = -3+3 = 0
pb
pc
CH.first
1 0
-1 4 ∧


pa
AH.first
1 0
-3 6
7 12 ∧
-1 4
3 6
-9 10
8 14 ∧
pb
BH.first
CH.first
1 0
-1 4
-9 10 ∧
pc


pa
AH.first
1 0
-3 6
7 12 ∧
-1 4
3 6
-9 10
8 14 ∧
pb
BH.first
CH.first
1 0
-1 4
-9 10
7 12 ∧
pc


pa
AH.first
1 0
-3 6
7 12 ∧
-1 4
3 6
-9 10
8 14 ∧
pb
p
BH.first
CH.first
1 0
-1 4
-9 10
7 12
8 14 ∧
pc

2.6 静态链表

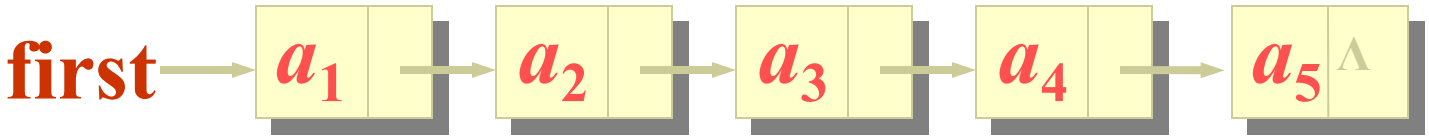
- 为数组中每一个元素附加一个链接指针，就形成静态链表结构。

- 静态链表每个结点由两个数据成员构成：**data**域存储数据，**link**域存放链接指针。

- 处理时可以不改变各元素的物理位置，只要重新链接就能改变这些元素的逻辑顺序。

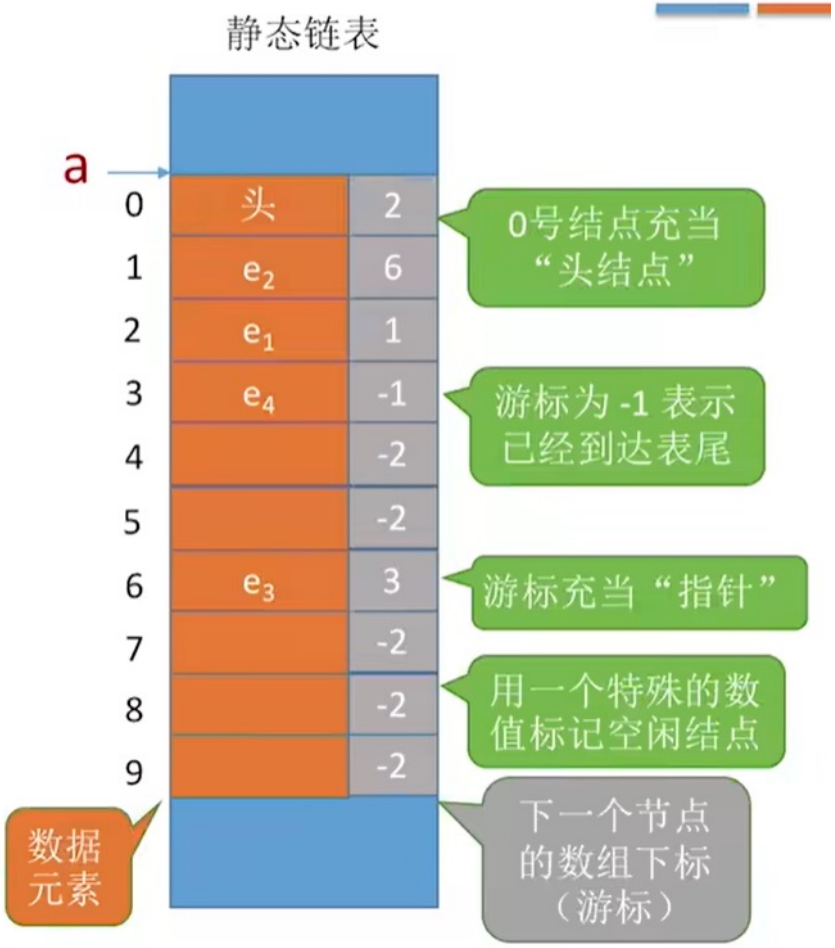
- 它是利用数组定义的，在整个运算过程中存储空间的大小不会变化。

静态链表的结构



	0	1	2	3	4	5
data		a_3	a_4	a_1	a_5	a_2
link	3	2	4	5	−1(0)	1

- 0号是表头结点，link给出首元结点地址。
- 循环链表收尾时link = 0，回到表头结点。
- 如果不是循环链表，收尾结点指针link = −1。
- link指针是数组下标，因此是整数。



静态链表：用数组的方式实现的链表

优点：增、删 操作不需要大量移动元素
缺点：不能随机存取，只能从头结点开始依次往后查找；容量固定不可变

适用场景：①不支持指针的低级语言；②数据元素数量固定不变的场景（如操作系统的文件分配表FAT）

提交时间：9 月 18 日，上午上课前提交

第 2 章：

(1) 第 83 页，2.2

(2) 第 84 页，2.3